

■ TEAM AND PROJECT

Fax Console

A house PBX's legacy fax tools, modernized with IBM Bob while the phone line kept working.

Track Modernize What Matters

Team Jeffrey Hein, solo

Built with IBM Bob

Repository github.com/jphein/fax-console

Live demo fax.realm.watch (replay mode, fixtures only)

Video [video.url](#)

■ TOOL USAGE & DEPTH OF ADOPTION · 30%

The rubric asks: "Did the team actually build with IBM Bob as a core part of their workflow, not just bolt it on at the end?"

Bob did the modernization, and every run is on record

1

Analyze

Module map, every shell-out, fragile patterns, a migration order.

`docs/analysis.md`

2

Package

The CLI as a typed package: pure logic split from I/O.

`faxcli/`

3

Test

Unit, golden and characterization tests on scrubbed fixtures.

`tests/`

4

Extract

The Fax panel out of the monolith, as a typed API.

`faxconsole/`

5

Isolate

One Asterisk adapter, with replay mode on fixtures.

`adapter + replay`

Bob factor

`bob.factor_pct`

of committed lines written by Bob

Bob-assisted commits

`bob.commits`

of `commits.in_window`, each marked
Assisted-by: IBM Bob

Bob runs on record

`bob.runs`

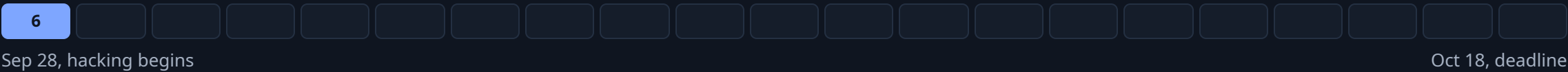
prompt, transcript and guard log
for each

Rejected in review

Run 4 sent PBX commands unquoted, so Asterisk would see only the first word. Run 5 fixed it, with a test.

How Bob runs. Bob Shell, headless, inside an OS sandbox. Claude (Anthropic) drives it and checks every claim against the code; Jeffrey approves at two checkpoints.

Bob from day one: every run, by day



DAY	RUNS	WHAT BOB DID	BOBCOINS
Sep 28	runs 0 to 5	Sandbox smoke test · Modernization analysis · Isolation check · Sandbox check · The faxcli package · The review fixes	14.40
So far	6 runs	on 1 of the 21 days	14.40

Rebuilt from the ledger in docs/bob-usage.md by docs/deck/timeline.py. Each run's prompt, transcript and guard log are in docs/bob-runs/. In run 5 the sandbox refused Bob's git stash.

The rubric asks: "Is the use case a credible, realistic application of IBM Bob (vs. a generic project that happens to touch it)?"

A realistic job for Bob: legacy code that runs a real phone line

The legacy system

- telephony-console.py: **11,858** lines in one file, with hand-rolled routing
- It shells out for every read: 11 call sites (analysis \$4)
- fax/cli.py: **288** lines around Asterisk SendFax, no tests
- In production on the house PBX: status, the fax log and sending must keep working

Real legacy, real stakes

Framework-free production code. The track's own list is the work: structure, tests, an API, modernization docs.

Safe by construction

Bob works in an OS sandbox with no network. In run 5 it tried `git stash`; the read-only `.git` refused it.

No private data reached an AI tool

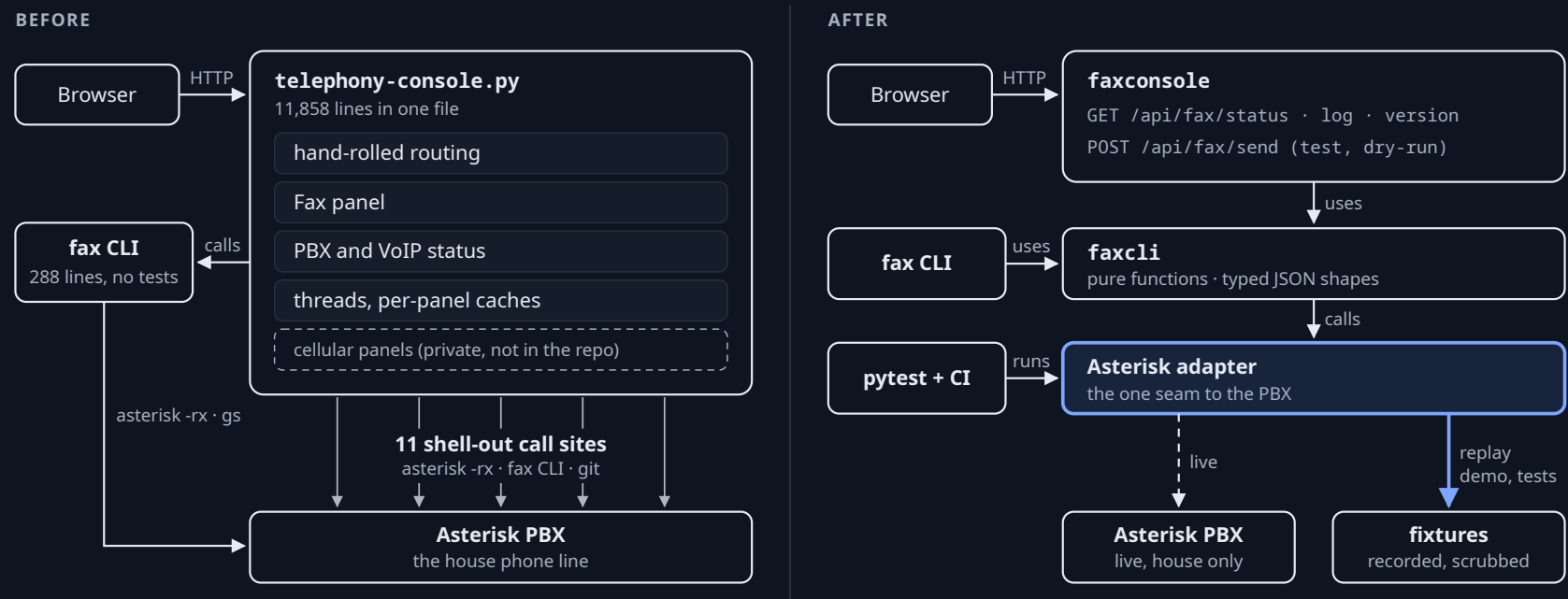
Every prompt passes the scrub gate first. Fixtures use 555-01xx numbers and example.com hosts.

The pattern carries over to any single-file admin tool: put one seam in front of the live system, record fixtures behind it, and the tool becomes testable and safe to demo.

■ FUNCTIONALITY & TECHNICAL EXECUTION · 30%

The rubric asks: “Does the project work as demoed? Is it technically sound given the 3-week timeframe?”

Before and after, and the proof it works



Before: each reader shells out on its own (11 call sites, analysis \$4), and nothing runs without the live line. After: one adapter is the only way in, and replay mode points it at recorded fixtures.

Tests <code>tests.count</code> on scrubbed fixtures	Line coverage <code>tests.coverage_pct</code> faxcli, faxconsole	CI on every push <code>ci.status</code> GitHub Actions, scrub gate	Live demo fax.realm.watch replay mode, read-only
--	---	---	--

The rubric asks: "Can the team clearly explain what they built and how Bob was used in a few minutes?"

In one minute, and how to check it yourself

What we built

A house PBX's legacy fax tools, modernized without taking the phone line down: a typed, tested `faxcli` package, and the Fax panel as a small typed API (`faxconsole`) behind one Asterisk adapter, with a replay mode.

How Bob was used

Bob wrote the analysis, the package and its tests, and fixed most of what review found (Bob factor `bob.factor_pct`), headless, inside an OS sandbox, with every run recorded.

Improvements made in the window

Commit #1 is the pre-hackathon baseline, scrubbed and otherwise unchanged, rebuilt once on day one. Everything after it is new. Bob wrote most of the code and tests. Claude wrote the scrub gate, CI, the sandbox and the ledger, plus a few small review fixes, each in its own commit with a test.

View it

Live demo (replay mode, read-only): <https://fax.realm.watch>
Demo video (under 3 minutes): [video.url](#)
The Bob ledger: `docs/bob-usage.md`
Each Bob run, prompt to guard log: `docs/bob-runs/`

Run it (no PBX needed)

```
$ git clone https://github.com/jphein/fax-console
$ cd fax-console
$ python3 -m faxconsole --replay tests/fixtures
  then open http://127.0.0.1:8093
$ python3 -m pytest -q
```

See which commits Bob helped write

```
$ git log --oneline --grep 'Assisted-by: IBM Bob'
```

What we learned, and what's next

LEARNED

- **Scrub before the first prompt.** The baseline was cleaned of numbers, names and hosts before any AI tool saw it, and a pre-commit check keeps it that way.
- **Review Bob against the code.** The analysis's most important finding, a PBX that cannot be read reported as `ok: true`, came from review, not from Bob.
- **learn.3**

ROOM TO IMPROVE

- **improve.1**
- **improve.2**

NEXT

- Deploy the inbound-fax path on a test line that cannot disturb the house number.
- Bring the rest of the console behind the same seam.

NEXT: LTE ON CBRS

A Magma core on CBRS Band 48 shared spectrum, linked over the SGs interface to a circuit-switched core, so LTE phones can send SMS and fall back to voice calls through the same PBX. Idea stage, not built.